Ответы на контрольные вопросы ПР7 (№ 11–12)

Индивидуальный проект «Сервис планирования киносъёмок» (база данных `data/kinoplan.sqlite`)

Лимонов Никита Андреевич, ЭФБО-08-24

11. Для чего используется WHERE?

`WHERE` фильтрует строки: в результат попадают только те, для которых условие истинно. Строки, где условие ложно или равно `NULL`, отбрасываются. В условии используются сравнения (=, <>, >= и др.), `BETWEEN`, `IN`, `LIKE`, `IS NULL`, а объединяются условия через `AND`, `OR`, `NOT`.

Примеры из проекта:

Задача	Условие WHERE
Локации для группы от N человек (`/locations/?capacity=N`)	`WHERE capacity >= ?`
Занята ли локация в дату	`WHERE location_id = ? AND shooting_date = ? AND is_cancelled = 0`
Свободные локации на дату (`/locations/free/`)	`WHERE capacity >= ? AND id NOT IN (SELECT location_id FROM shootings WHERE shooting_date = ? AND is_cancelled = 0)`
Съёмки в ноябре	`WHERE shooting_date BETWEEN '2026-11-01' AND '2026-11-30'`
Сцены без назначенной съёмки	`LEFT JOIN shootings … WHERE shootings.id IS NULL`

До ПР7 такую фильтрацию делал Python: загружал все локации из JSON и перебирал их генератором `filter_locations_by_capacity()`. Теперь условие выполняет SQLite и возвращает только нужные строки.

Значения от пользователя в `WHERE` подставляются только параметрами `?`, а не f-строкой. Тогда введённый текст не может изменить смысл запроса:

```
conn.execute(
    "SELECT id, name, capacity FROM locations WHERE capacity >= ?",
    [crew_size],
)
```

WHERE и HAVING. `WHERE` работает с отдельными строками до группировки, поэтому агрегатные функции в нём использовать нельзя. `HAVING` фильтрует уже сформированные группы. В статистике загруженных локаций нужны оба:

```
SELECT locations.name, COUNT(*) AS shootings_count
FROM shootings
JOIN locations ON shootings.location_id = locations.id
WHERE shootings.is_cancelled = 0   -- строки: без отменённых
GROUP BY locations.id, locations.name
HAVING COUNT(*) >= 2;              -- группы: от 2 съёмок
```

`WHERE` работает также в `UPDATE` и `DELETE`: без него изменятся или удалятся все строки таблицы. В скрипте импорта `DELETE FROM shootings` написан без `WHERE` намеренно, чтобы очистить таблицу перед повторным переносом.

12. Чем отличаются операторы =, <> и !=?

- = означает «равно». В SQL сравнение записывается одним знаком = (в Python это ==).
- <> и != означают «не равно», результат у них одинаковый. Разница только в происхождении: <> входит в стандарт SQL и работает в любой СУБД, != — распространённое расширение (SQLite, PostgreSQL, MySQL). Для переносимости лучше писать <>.

Проверка в SQLite CLI на базе проекта. Поле `is_cancelled` принимает только 0 или 1 (`CHECK (is_cancelled IN (0, 1))`), поэтому все три условия отбирают одни и те же 6 активных съёмок из 7:

```
SELECT COUNT(*) FROM shootings WHERE is_cancelled = 0;    -- 6
SELECT COUNT(*) FROM shootings WHERE is_cancelled <> 1;   -- 6
SELECT COUNT(*) FROM shootings WHERE is_cancelled != 1;   -- 6
SELECT 1 <> 2, 1 != 2;                                    -- 1 | 1
```

Важно: `NULL`. Ни =, ни <>, ни != не сравнивают с `NULL`: результат сравнения равен `NULL` («неизвестно»), и строка не попадает в выборку ни по условию «равно», ни по «не равно».

```
SELECT NULL = NULL, NULL <> 1, NULL IS NULL;   -- NULL | NULL | 1
```

В проекте поля `films.director` и `locations.address` могут быть `NULL`. Условие `WHERE director <> 'А. Сергеев'` не вернёт фильм без режиссёра, хотя формально режиссёр у него «другой». Для отсутствующих значений есть отдельные операторы `IS NULL` и `IS NOT NULL` (запрос 2.5 в `sql/queries.sql`: `WHERE address IS NULL`).

Ещё одна особенность: в SQLite = сравнивает строки с учётом регистра, поэтому `'Вокзал' = 'вокзал'` даёт 0. Даты хранятся строками `ГГГГ-ММ-ДД`, и для них = работает правильно: `shooting_date = '2026-11-10'`.